# 📌 DAY 27: ERROR HANDLING IN JAVASCRIPT (TRY-CATCH-FINALLY) 🚨

Lakshmiprasanna Vakada

JavaScript applications can run into errors, but instead of breaking the program, we can handle them gracefully using try...catch...finally. Let's dive into error handling! 🔥

## 🔹 Why Handle Errors?

Errors can occur due to:

- ✅ Incorrect user input
- ✅ Network failures
- ✅ Undefined variables or properties
- ✅ Syntax issues

By handling errors properly, we can:

- Prevent unexpected crashes
- Show user-friendly error messages
- Debug issues effectively

- try...catch Syntax

```
try {
  // Code that may cause an error
} catch (error) {
  // Handle the error here
} finally {
  // (Optional) Code that always runs
}
```

# Using try...catch in Action

```
try {
  let result = 10 / 0;
  console.log(result); // Infinity (No
error)

  let x = y; // 'y' is not defined, will
throw an error
} catch (error) {
  console.log("An error occurred:",
error.message);
}
```

- If an error occurs inside try, the catch block executes.
- If no error occurs, catch is skipped.

# 🔹 The finally Block

The finally block always runs, whether an error occurs or not.

```
try {
  console.log("Try block executed");
} catch (error) {
  console.log("Error caught");
} finally {
  console.log("Finally block executed");
}
```

✅ Useful for cleanup tasks like closing connections or resetting values.

# Throwing Custom Errors

You can manually throw errors using throw.

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by
zero");
  }
  return a / b;
}

try {
  console.log(divide(10, 0));
} catch (error) {
  console.log("Error:", error.message);
}
```

# Handling Different Types of Errors

JavaScript has built-in error types like:

- ReferenceError – Using an undeclared variable
- TypeError – Performing invalid operations on data types
- SyntaxError – Incorrect syntax (only caught in eval())
- RangeError – Exceeding numeric limits

```
try {
  let num = undefined;
  console.log(num.toUpperCase());
 // TypeError
} catch (error) {
  console.log("Error Type:", error.name);
  console.log("Error Message:",
error.message);
}
```

# Nested try...catch (Handling Multiple Errors)

```
try {
  try {
    let arr = [1, 2];
    console.log(arr[5].toString()); //
TypeError
  } catch (error) {
    console.log("Inner Error:",
error.message);
  }

  let result = 10 / 0;
} catch (error) {
  console.log("Outer Error:",
error.message);
}
```

✅ If an inner catch block handles the error, the outer catch won't execute.